

Lighting and Shading



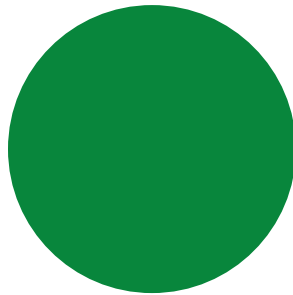
Department of
Informatics



Graphics, Interaction,
and Games Lab

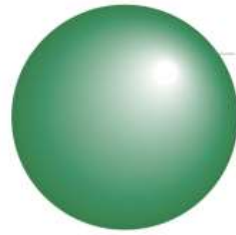
Mengapa kita butuh lighting dan shading?

- Misal kita membangun sebuah model sphere menggunakan polygon dan memberinya warna menggunakan `glColor`. Hasil yang kita dapat adalah sbb:
- Padahal, hasil yang diinginkan

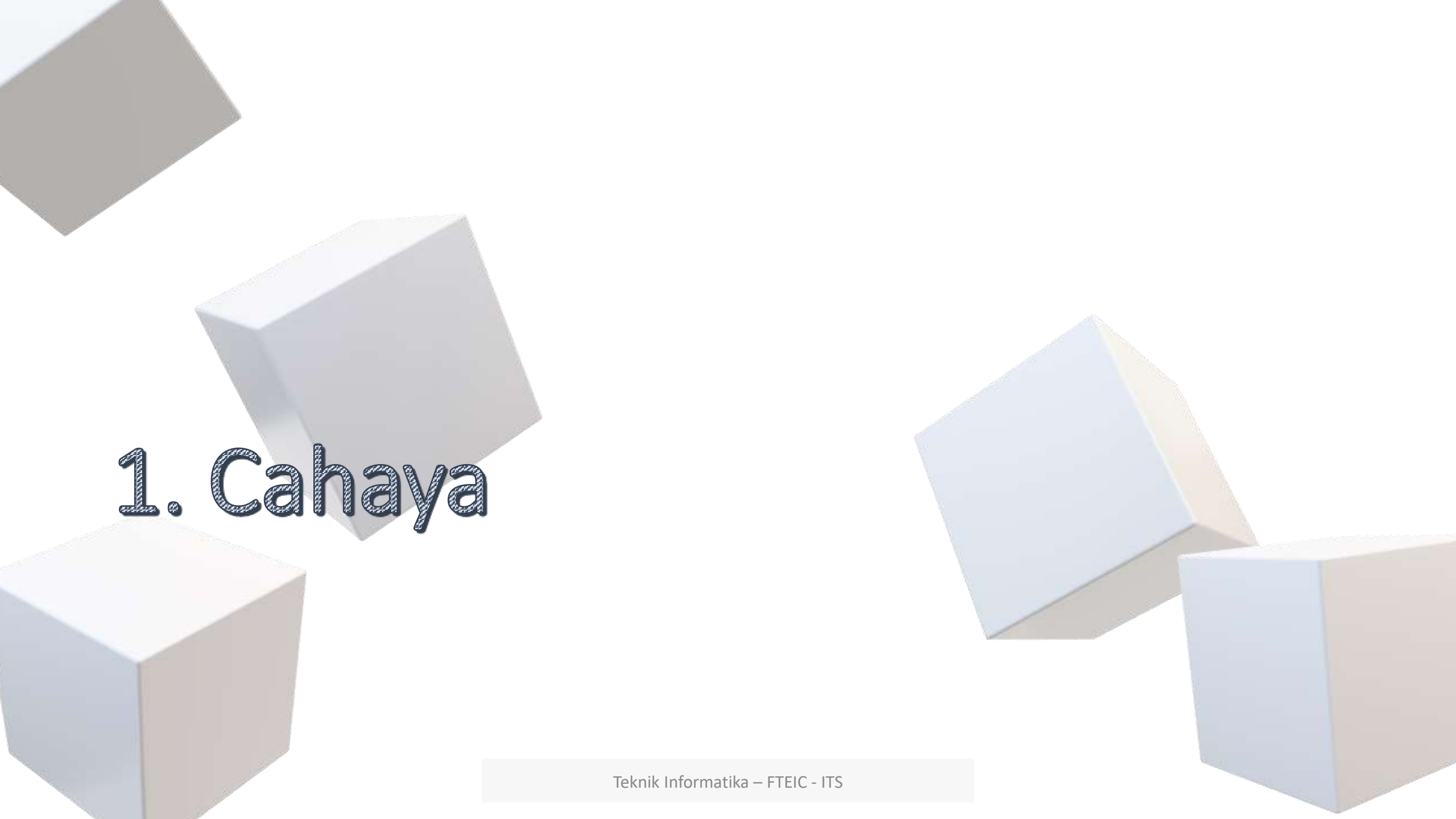


Bagaimana sphere kelihatan nyata?

- Sphere yang nyata gambarnya adalah seperti berikut:



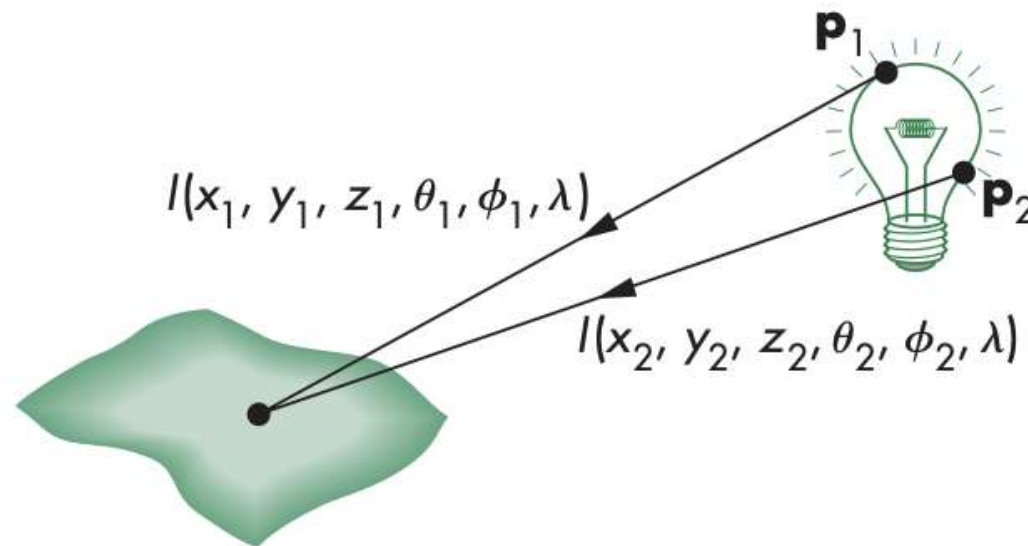
- Interaksi antara cahaya dengan material mengakibatkan masing-masing point mempunyai warna atau bayangan (shade) yang berbeda.
- Disini kita harus memperhatikan:
 - Sumber cahaya (Light sources)
 - Sifat material (Material properties)
 - Lokasi dari viewer (Location of viewer), dan
 - Arah dari surface (Surface orientation)



1. Cahaya

Light Sources (Sumber Cahaya)

Sumber cahaya yang umum sangat sulit untuk disimulasikan karena kita harus mengintegrasikan cahaya yang dari semua titik pada source.

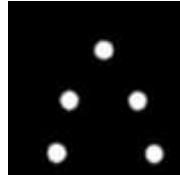


Terdapat 4 Macam Sumber Cahaya



Ambient light

Intensitas cahayanya (I_a) sama pada semua titik.



Point source

Intensitas cahaya yang diterima oleh suatu permukaan proporsional terhadap jarak antara point source dengan permukaan.



Spotlight

Cahaya hanya dipancarkan pada cakupan sudut tertentu saja.

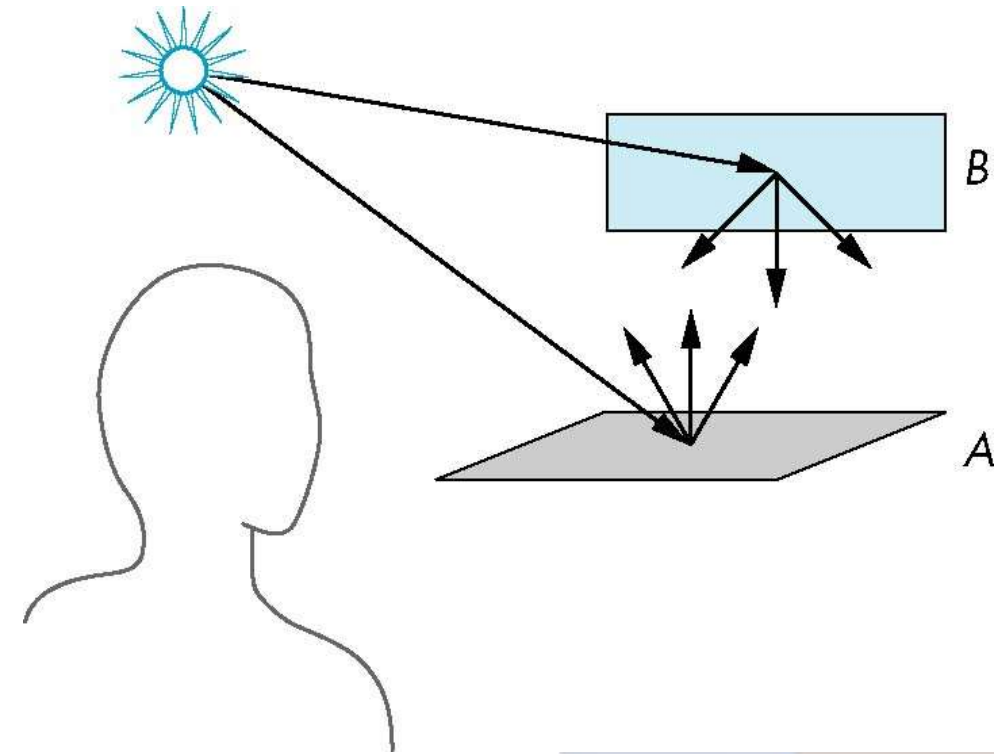


Distant light

Distant source = cahaya berasal dari jarak yang jauh sehingga arah cahayanya parallel

Scattering (Penyebaran Cahaya)

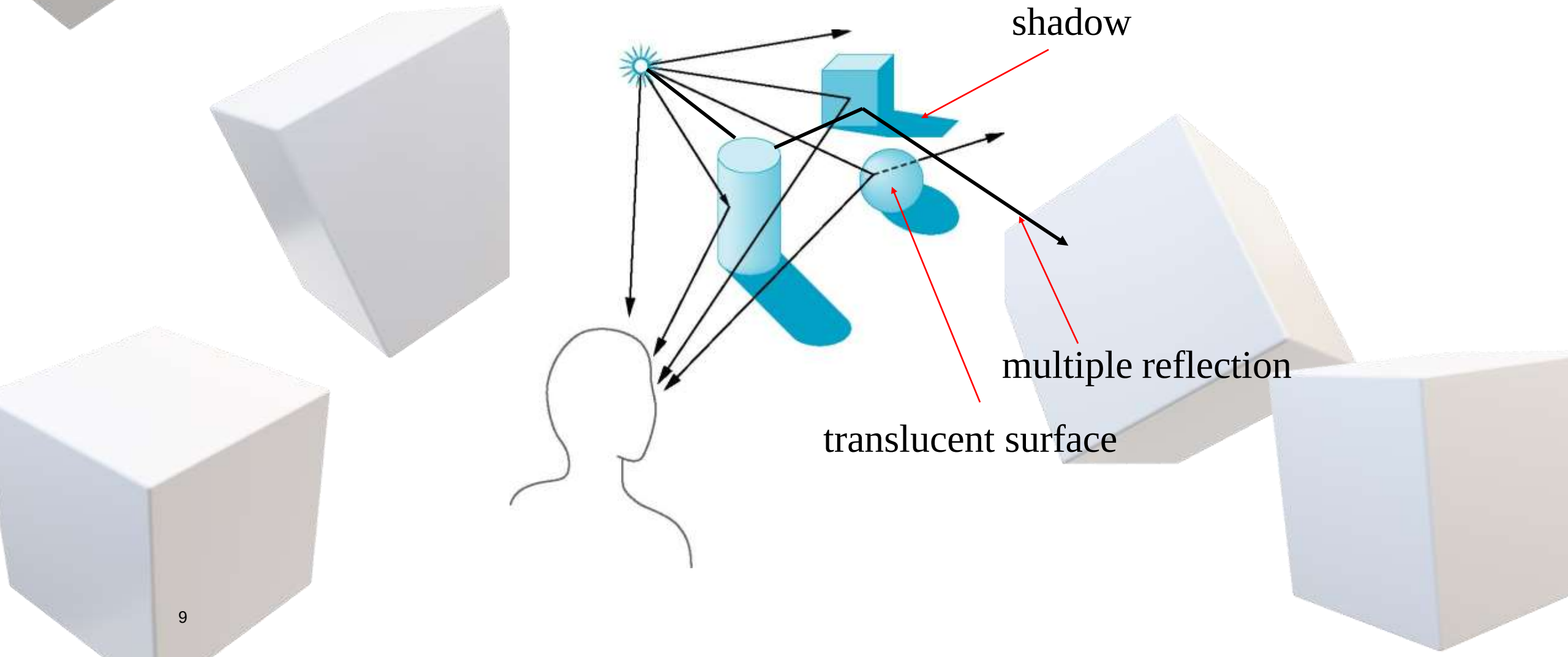
- Ketika cahaya mengenai sisi A, maka:
 - Beberapa disebarkan dan
 - Beberapa diserap
- Sebagian cahaya yang disebarkan tadi, mengenai B dan
 - Beberapa disebarkan
 - Beberapa diserap
- Sebagian cahaya yang disebarkan tadi, Sebagian mengenai A dan seterusnya



Rendering Equation

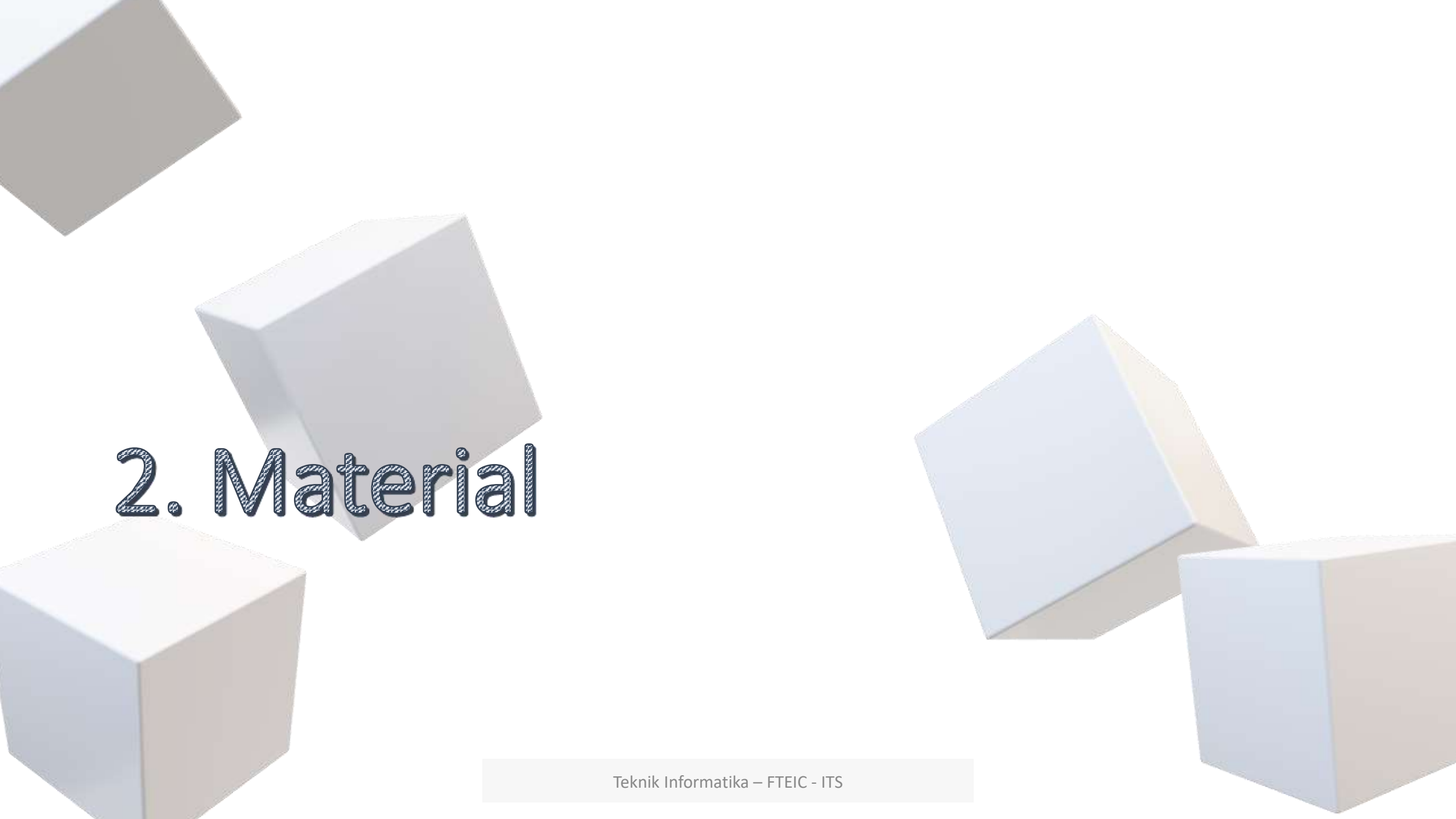
- Proses penyebaran dan penyerapan cahaya yang tiada henti tadi kemudian dideskripsikan dalam persamaan rendering (*rendering equation*)
 - Yang sebenarnya secara umum tidak dapat diselesaikan
 - Ray tracing merupakan kasus khusus ketika semua cahaya dipantulkan
- Rendering equation bersifat umum dan meliputi:
 - Bayangan (Shadow)
 - Penyebaran banyak cahaya dari objek ke objek

Contoh Efek Globalnya



Local vs Global Rendering

- Shading yang benar membutuhkan perhitungan global yang melibatkan semua objek dan sumber cahaya
 - Tetapi hal ini tidak cocok dengan pipeline model yang bayangan setiap poligonnya dirender secara independent (local rendering)
- Namun, di dalam computer graphics, khususnya real time graphics, kita inginnya melihat sesuatu “look right”
 - Untuk itu, terdapat beberapa teknik agar menyerupai global effects



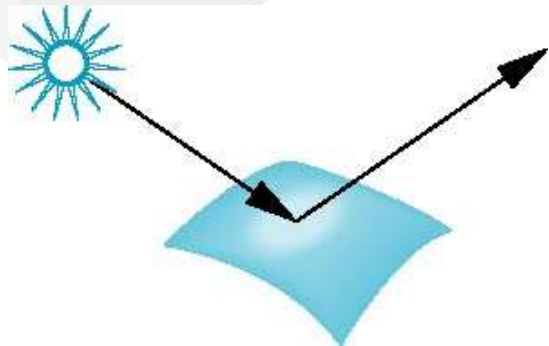
2. Material

Interaksi antara Cahaya-Material

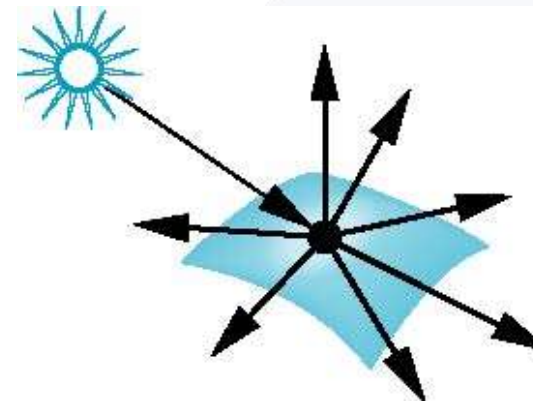
- Cahaya yang mengenai suatu objek sebagian diserap dan Sebagian disebar (dipantulkan).
- Kapasitas cahaya yang dipantulkan mempengaruhi warna dan tingkat kemengkilapan (brightness) suatu objek
 - Contoh: sebuah permukaan kelihatan berwarna merah dibawah cahaya putih karena komponen warna merah pada cahaya tersebut dipantulkan dan yang lainnya diserap.
- Cahaya yang dipantulkan menyebar dengan sifat yang tergantung pada tingkat kemulusan dan arah suatu permukaan.

Tipe Permukaan (Surface Types)

- Semakin halus suatu permukaan, cahaya yang dipantulkan akan semakin terkonsentrasi pada satu arah
- Permukaan yang kasar, akan menyebarkan cahaya pada semua arah.



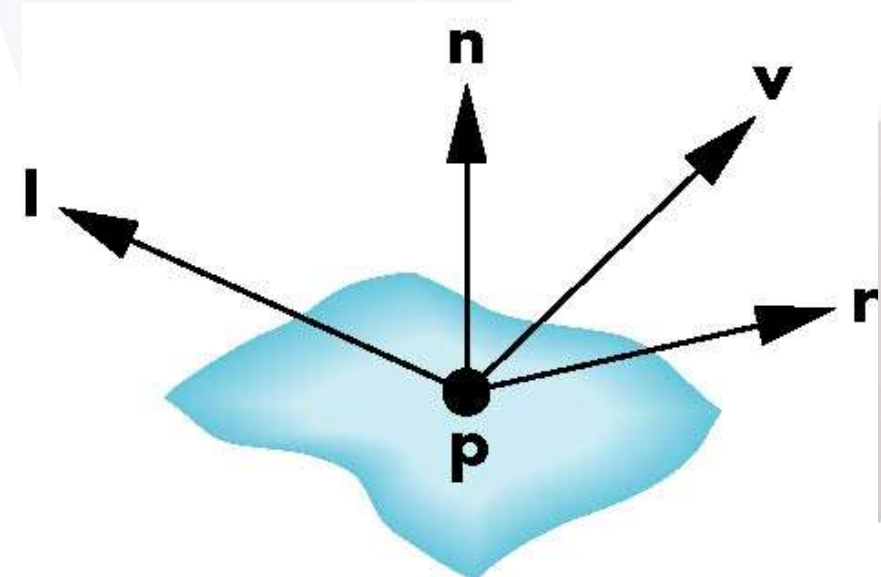
smooth surface



rough surface

Phong Model

- Phong Model merupakan suatu model sederhana untuk mensimulasikan interaksi antara cahaya dengan material yang bisa dihitung secara cepat
- Phong model mempunyai 3 komponen:
 - Diffuse
 - Specular
 - Ambient
- Phong model menggunakan 4 vektor:
 - To source (I)
 - To viewer (v)
 - Normal (n)
 - Perfect reflector (r)



1. Lambertian (Rough/diffuse) Surface

- Merupakan suatu permukaan yang secara sempurna menyebarkan pantulan cahaya
- Cahaya disebarkan ke segala arah dengan proporsi yang sama
- Jumlah dari cahaya yang dipantulkan proporsional dengan jumlah cahaya yang datang. Sifat cahayanya mengikuti Lambert law, jika q_i adalah sudut antara cahaya yang datang dengan normal pada titik tertentu, maka
 - Cahaya yang dipantulkan $\sim \cos q_i$
 - $\cos q_i = \mathbf{l} \cdot \mathbf{n}$, $\mathbf{l}$ adalah arah cahaya yang datang dan $\mathbf{n}$ adalah vector normal
 - Jika k_d adalah koefisien refleksi, maka intensitas cahaya yang didifusikan adalah

$$I_d = k_d(\mathbf{l} \cdot \mathbf{n})L_d.$$

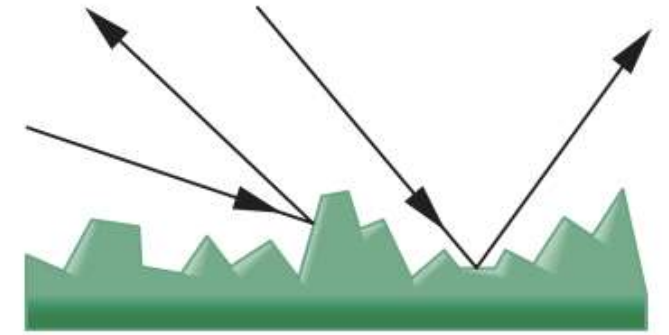
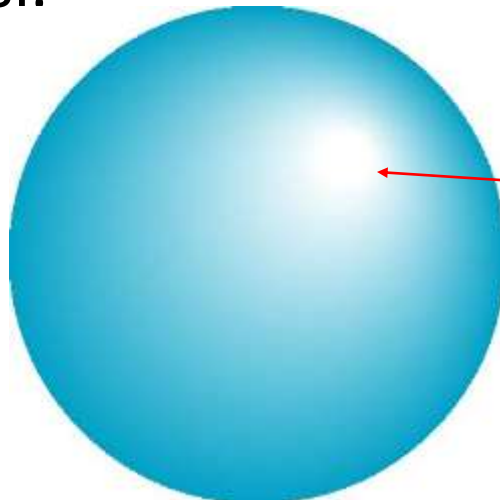


FIGURE 6.14 Rough surface.

2. Specular Surface

- Sebagian besar permukaan adalah salah satu ideal diffusers atau perfectly specular (ideal reflectors).
- Specular surface, permukaannya halus sehingga memantulkan cahaya secara sempurna
- Permukaan yang halus akan menampilkan highlight karena cahaya yang datang akan dipantulkan ke arah yang dekat dengan lokasi dimana perfect reflection terkonsentrasi.



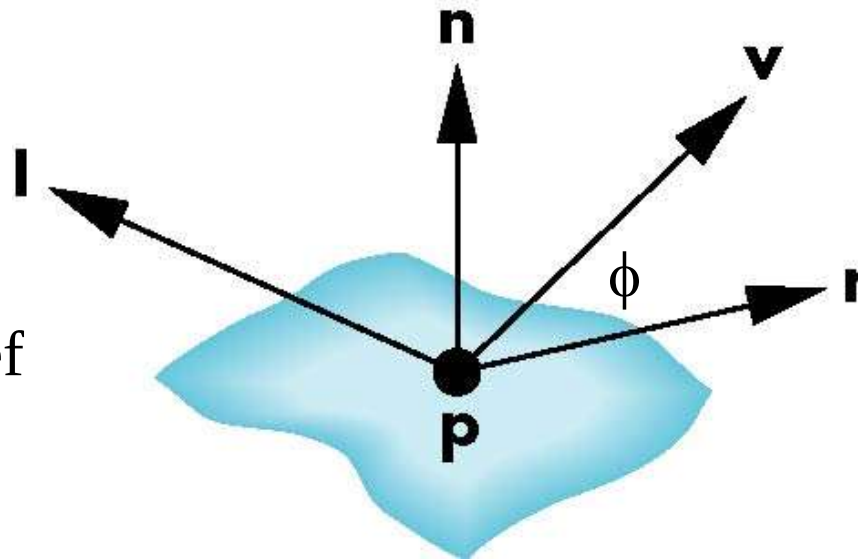
specular
highlight

Specular Reflections pada Phong Model

- Di dalam Phong model dinyatakan bahwa jumlah cahaya yang dilihat oleh viewer tergantung pada sudut antara viewer dengan r , ϕ .

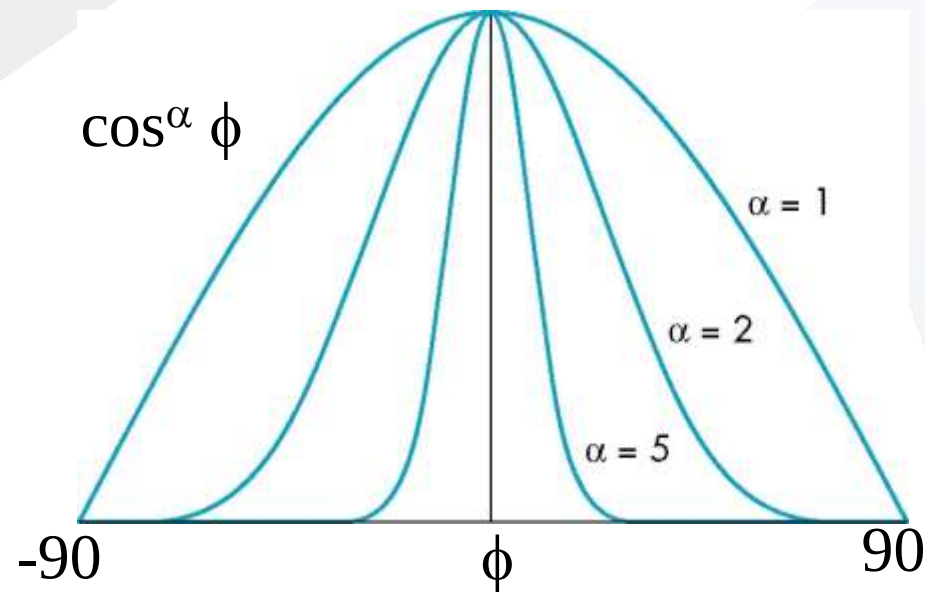
$$I_r \sim k_s I \cos^{\alpha} \phi$$

reflected intensity shininess coef
incoming intensity
absorption coef



The Shininess Coefficient (Tingkat berkilaunya suatu material)

- Nilai α antara 100 dan 200 untuk benda yang terbuat metal
- Nilai α antara 5 dan 10 memberikan permukaan yang tampak seperti plastik



3. Ambient Light di dalam Phong Model

- Ambient light merupakan hasil interaksi dari banyak sumber cahaya dengan objek-objek di dalam lingkungan.
- Jumlah dan warnanya tergantung pada warna cahaya dan karakteristik material dari objek
- Pada Phong model, tambahkan $k_a I_a$ ke term diffuse dan seto diffuse and specular terms

reflection coef

intensity of ambient light

Sumber Cahaya pada Phong Model

- Pada Phong Model, kita menambahkan hasil dari masing-masing light source.
- Masing-masing light source mempunyai term untuk diffuse, specular, dan ambient untuk membuatnya lebih fleksibel
- Komponen red, green dan blue yang terpisah.
- Sehingga, terdapat 9 koefisien untuk masing-masing point source
 - $I_{dr}, I_{dg}, I_{db}, I_{sr}, I_{sg}, I_{sb}, I_{ar}, I_{ag}, I_{ab}$

Material Properties pada Phong Model

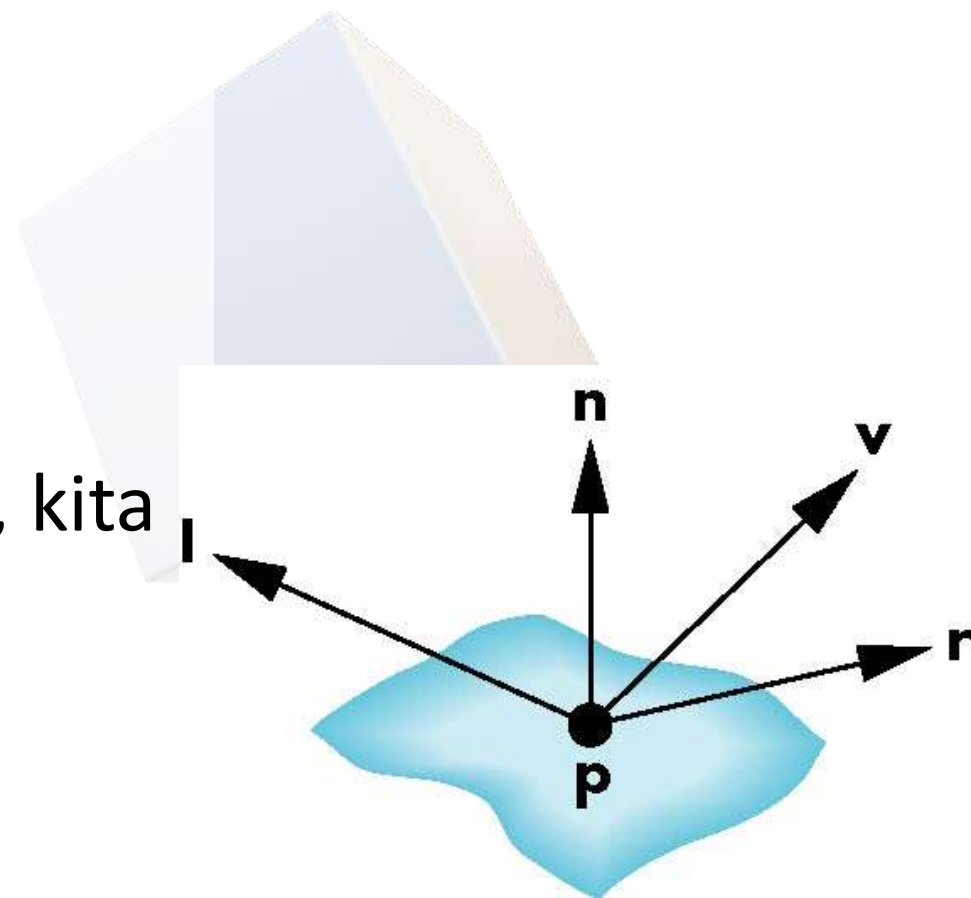
- Material properties sesuai dengan property untuk light source
 - Terdapat 9 koefisien untuk absorbtion karena masing-masing warna mempunyai koefisien sendiri
 - $k_{dr}, k_{dg}, k_{db}, k_{sr}, k_{sg}, k_{sb}, k_{ar}, k_{ag}, k_{ab}$
 - Shininess coefficient α

Menambahkan Semua Komponen

- Untuk masing-masing light source dan komponen warna, Phong model dapat ditulis (tanpa term untuk jarak) sbb

$$I = k_d I_d \mathbf{l} \cdot \mathbf{n} + k_s I_s (\mathbf{v} \cdot \mathbf{r})^\alpha + k_a I_a$$

- Untuk masing-masing komponen warna, kita menambahkan kontribusi untuk semua sumber



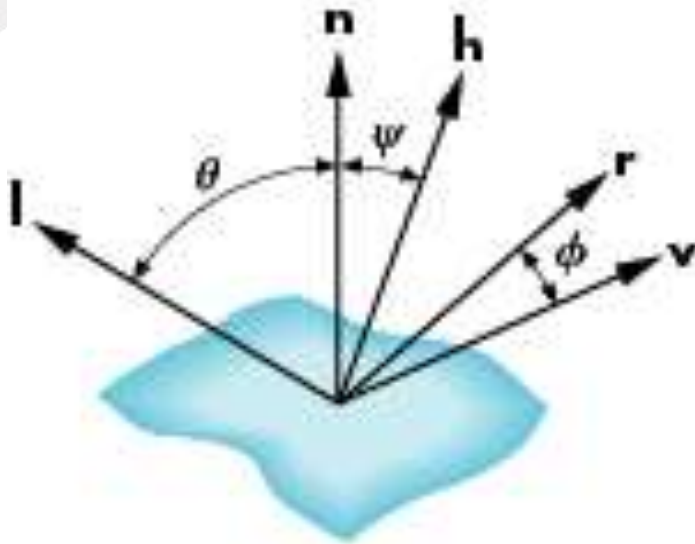
Memodifikasi Phong Model

- Term specular pada model Phong membuat problematic karena membutuhkan perhitungan untuk setiap vector refleksi (r) and vector view (v) untuk setiap vertex.
- Blinn menyarankan untuk menggunakan pendekatan menggunakan 'halfway vector' agar lebih efisien.

Apa itu Halfway Vector?

- **h** merupakan normalized vector halfway antara **l** dan **v**

$$\mathbf{h} = (\mathbf{l} + \mathbf{v}) / |\mathbf{l} + \mathbf{v}|$$



Sehingga Term Specular menggunakan halfway vector adalah sbb:

- Ganti $(\mathbf{v} \cdot \mathbf{r})^\alpha$ dengan $(\mathbf{n} \cdot \mathbf{h})^\beta$
- β dipilih agar match dengan shininess
- Catatan: halfway angle adalah setengah dari sudut antara $\mathbf{r}$ dan $\mathbf{v}$ jika vector-vektornya adalah coplanar
- Model hasilnya disebut dengan modified Phong atau Phong-Blinn lighting model
 - Digunakan pada OpenGL standard

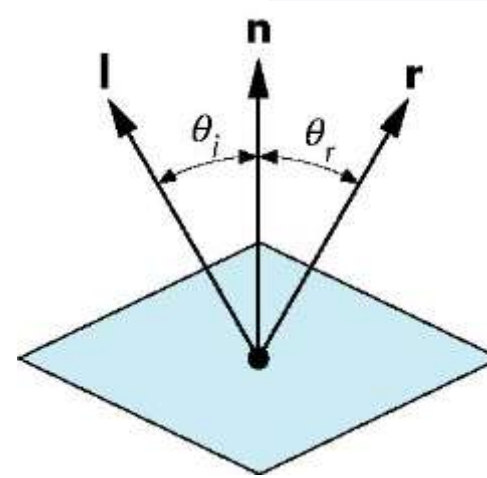
Perhitungan Vector

- $\mathbf{l}$ dan $\mathbf{v}$ sudah ditentukan di dalam aplikasi
- Computer menghitung $\mathbf{r}$ dari $\mathbf{l}$ dan $\mathbf{n}$
- Problemnya adalah menentukan $\mathbf{n}$
- OpenGL menyerahkan perhitungan normal pada aplikasi

Vektor Normal pada Pemantulan Sempurna

- Normal ditentukan oleh orientasi lokal, dimana:
 - Sudut dari cahaya yang datang (Angle of incidence) = sudut pantulan (angle of reflection)
- Disini, ke 3 vector tersebut harus coplanar

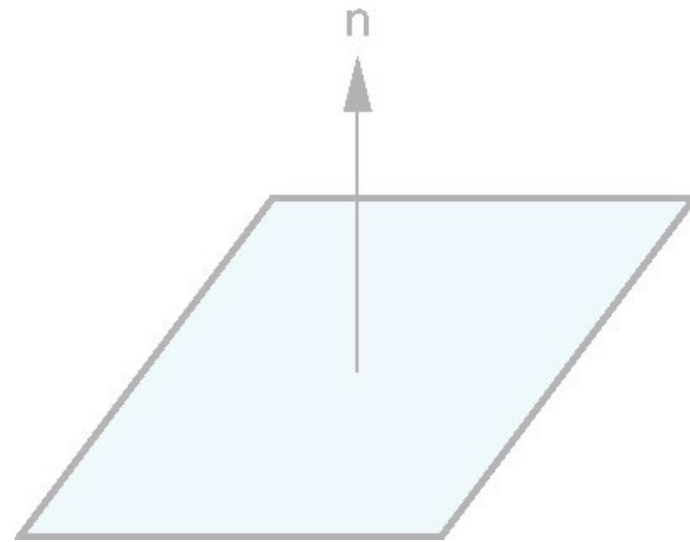
$$\mathbf{r} = 2 (\mathbf{l} \cdot \mathbf{n}) \mathbf{n} - \mathbf{l}$$



Bidang Normal

- Persmaan untuk suatu bidang adalah: $ax+by+cz+d = 0$
- Suatu bidang dinyatakan dengan 3 titik p_0, p_2, p_3 atau normal $\mathbf{n}$ dan p_0
- Normal dapat dihitung dengan:

$$\mathbf{n} = (p_2 - p_0) \times (p_1 - p_0)$$

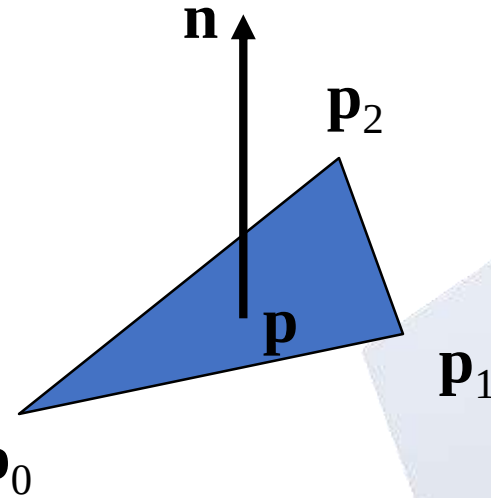


Contoh: Normal untuk segitiga

plane $\mathbf{n} \cdot (\mathbf{p} - \mathbf{p}_0) = 0$

$$\mathbf{n} = (\mathbf{p}_2 - \mathbf{p}_0) \times (\mathbf{p}_1 - \mathbf{p}_0)$$

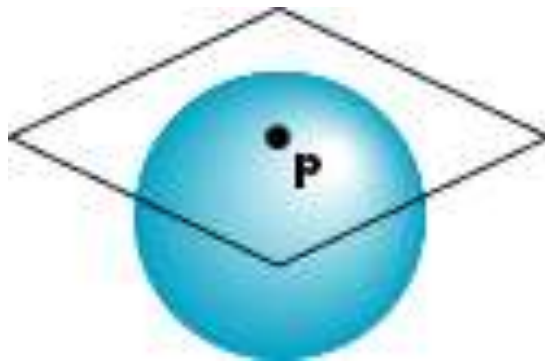
normalize $\mathbf{n} \leftarrow \mathbf{n} / |\mathbf{n}|$



Catatan: right-hand rule menentukan outward face

Normal pada Sphere

- Fungsi implisitnya $f(x,y,z)=0$
- Normal dihitung dengan gradian
- Untuk Sphere $f(\mathbf{p})=\mathbf{p}\cdot\mathbf{p}-1$
- $\mathbf{n} = [\partial f/\partial x, \partial f/\partial y, \partial f/\partial z]^T = \mathbf{p}$



Parametric Form

- For sphere

$$x=x(u,v)=\cos u \sin v$$

$$y=y(u,v)=\cos u \cos v$$

$$z= z(u,v)=\sin u$$

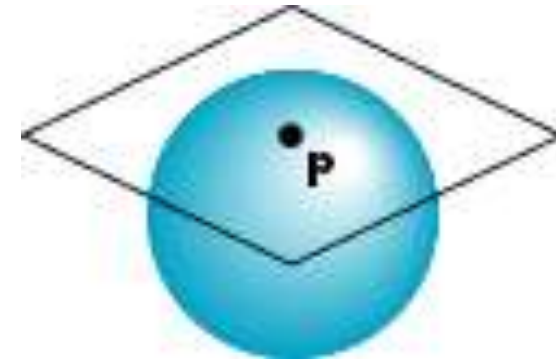
- Tangent plane determined by vectors

$$\partial \mathbf{p} / \partial u = [\partial x / \partial u, \partial y / \partial u, \partial z / \partial u]^T$$

$$\partial \mathbf{p} / \partial v = [\partial x / \partial v, \partial y / \partial v, \partial z / \partial v]^T$$

- Normal given by cross product

$$\mathbf{n} = \partial \mathbf{p} / \partial u \times \partial \mathbf{p} / \partial v$$



Lighting dan Shading pada WebGL



Department of
Informatics



Graphics, Interaction,
and Games Lab

Lighting di WebGL

- Dari sebelumnya kita dapatkan bahwa lighting membutuhkan
 - Vektor normal
 - Karakteristik material
 - Cahaya
- Bisa dihitung pada aplikasi ataupun shader

Normalization

- Term cosine pada perhitungan lighting dapat dihitung menggunakan *dot product*
- Penggunaan “unit length vectors” membuat perhitungan lebih sederhana
- GLSL mempunyai fungsi normalization

Sumber Cahaya, Jarak, dan Arahnya

- Warna sumber cahaya, ditentukan dengan RGBA
- Posisinya dinyatakan dalam homogeneous coordinates
 - Jika $w = 1.0$, maka posisi dari suatu titik yang jelas
 - Jika $w = 0.0$, maka sumber cahaya tersebut dari jauh

Pergerakan Light Sources

- Sumber cahaya adalah objek geometris yang posisi atau arahnya dipengaruhi oleh model-view matrix
- Tergantung di mana kita melakukan setting pada posisi (arah), kita dapat:
 - Menggerakkan light source(s) dengan object(s)
 - Lokasi object(s) fix dan menggerakkan light source(s)
 - Lokasi light source(s) fix dan menggerakkan object(s)
 - Menggerakkan light source(s) dan object(s) secara independent

Light Properties

- Untuk masing-masing light source, kita dapat mengeset RGBA untuk komponen-komponen diffuse, specular, dan ambient, serta dari posisinya:

```
var lightPosition = vec4(1.0, 1.0, 1.0, 0.0 );  
var lightAmbient = vec4(0.2, 0.2, 0.2, 1.0 );  
var lightDiffuse = vec4( 1.0, 1.0, 1.0, 1.0 );  
var lightSpecular = vec4( 1.0, 1.0, 1.0, 1.0 );
```

Karakteristik Material

- Material properties harus match dengan term yang ada pada light model
- Komponen w menyatakan opacity

```
var materialAmbient = vec4( 1.0, 0.0, 1.0, 1.0 );  
var materialDiffuse = vec4( 1.0, 0.8, 0.0, 1.0);  
var materialSpecular = vec4( 1.0, 0.8, 0.0, 1.0 );  
var materialShininess = 100.0;
```

Using MV.js for Products

```
var ambientProduct = mult(lightAmbient, materialAmbient);  
var diffuseProduct = mult(lightDiffuse, materialDiffuse);  
var specularProduct = mult(lightSpecular, materialSpecular);
```

```
gl.uniform4fv(gl.getUniformLocation(program,  
    "ambientProduct"),    flatten(ambientProduct));  
gl.uniform4fv(gl.getUniformLocation(program,  
    "diffuseProduct"),    flatten(diffuseProduct) );  
gl.uniform4fv(gl.getUniformLocation(program,  
    "specularProduct"),    flatten(specularProduct) );  
gl.uniform4fv(gl.getUniformLocation(program,  
    "lightPosition"),    flatten(lightPosition) );  
gl.uniform1f(gl.getUniformLocation(program,  
    "shininess"),materialShininess);
```

Adding Normals for Quads

```
function quad(a, b, c, d) {  
    var t1 = subtract(vertices[b], vertices[a]);  
    var t2 = subtract(vertices[c], vertices[b]);  
    var normal = cross(t1, t2);  
    var normal = vec3(normal);  
    normal = normalize(normal);  
  
    pointsArray.push(vertices[a]);  
    normalsArray.push(normal);  
    .  
    .  
    .  
}
```

Per Vertex and Per Fragment Shaders



Department of
Informatics



Graphics, Interaction,
and Games Lab

Vertex Lighting Shaders I

```
// vertex shader
```

```
in vec4 aPosition;  
in vec4 aNormal;  
out vec4 vColor;  
uniform vec4 uAmbientProduct, uDiffuseProduct,  
            uSpecularProduct;  
uniform mat4 uModelViewMatrix;  
uniform mat4 uProjectionMatrix;  
uniform vec4 uLightPosition;  
uniform float uShininess;  
void main()  
{
```

Vertex Lighting Shaders II

```
vec3 pos = -(uModelViewMatrix * aPosition).xyz;  
vec3 light = uLightPosition.xyz;  
vec3 L = normalize( light - pos );  
vec3 E = normalize( -pos );  
vec3 H = normalize( L + E );
```

```
// Transform vertex normal into eye coordinates
```

```
vec3 N = normalize( (uModelViewMatrix*aNormal).xyz);
```

```
// Compute terms in the illumination equation
```

Vertex Lighting Shaders III

// Compute terms in the illumination equation

vec4 ambient = uAmbientProduct;

float Kd = max(dot(L, N), 0.0);

vec4 diffuse = Kd*uDiffuseProduct;

float Ks = pow(max(dot(N, H), 0.0), uShininess);

vec4 specular = Ks * uSpecularProduct;

if(dot(L, N) < 0.0) specular = vec4(0.0, 0.0, 0.0, 1.0);

gl_Position = uProjection * uModelView * aPosition;

color = ambient + diffuse + specular;

color.a = 1.0;

}

Vertex Lighting Shaders IV

```
// fragment shader
```

```
precision mediump float;
```

```
in vec4 vColor;
```

```
out vec4 fColor;
```

```
void main()
```

```
{
```

```
    fColor = vColor;
```

```
}
```

Fragment Lighting Shaders I

```
// vertex shader
```

```
in vec4 aPosition;  
in vec4 aNormal;  
out vec3 vN, vL, vE;  
uniform mat4 uModelViewMatrix;  
uniform mat4 uProjectionMatrix;  
uniform vec4 uLightPosition;
```

Fragment Lighting Shaders II

```
void main()
{
    vec3 pos = -(uModelViewMatrix * aPosition).xyz;
    vec3 light = uLightPosition.xyz;
    vec3 vL = normalize( light - pos );
    vec3 vE = -pos;
    vec3 vN = normalize( (uModelViewMatrix*aNormal).xyz);
    gl_Position = uProjectionMatrix * uModelViewMatrix * aPosition;
};
```

Fragment Lighting Shaders III

```
// fragment shader
```

```
precision mediump float;
```

```
uniform vec4 uAmbientProduct;
```

```
uniform vec4 uDiffuseProduct;
```

```
uniform vec4 uSpecularProduct;
```

```
uniform float uShininess;
```

```
in vec3 uN, uL, uE;
```

```
out vec4 fColor;
```

```
void main()  
{
```

Fragment Lighting Shaders IV

```
vec4 vColor;  
vec3 H = normalize( vL + vE );  
vec4 ambient = uAmbientProduct;  
float Kd = max( dot(vL, vN), 0.0 );  
vec4 diffuse = Kd*uDiffuseProduct;  
float Ks = pow( max(dot(N, H), 0.0), uShininess );  
vec4 specular = Ks * uSpecularProduct;  
    if( dot(L, N) < 0.0 ) specular = vec4(0.0, 0.0, 0.0, 1.0);  
fColor = ambient + diffuse +specular;  
fColor.a = 1.0;  
}
```

Teapot Examples



Vertex Lighting



Fragment Lighting

Latihan

- Kembangkan Tugas 1 kalian dengan menambahkan:
 - Fitur dengan interaksi pengguna untuk warna cahaya dari masing-masing sumber cahaya (ambient, specular, diffuse)
 - Posisi sumber cahaya bisa dirubah-rubah
 - Menggunakan fragment shader